Hardware Hacking & Embedded Security

A Zero-to-Professional Roadmap

5	40+	10+	800–1400 EGP
Phases	Weeks	Projects	Starter Budget

01 Foundations

02 Debug Interfaces

03 Firmware & RE

04 RF & Wireless

05 Advanced / Pro

How to Use This Roadmap

This roadmap is your **master reference** for going from zero to professional in hardware hacking and embedded security. It is designed around real hardware, real techniques, and tools you can buy in Egypt today. Every phase has a clear goal, a week-by-week study plan, hands-on projects, common mistakes to avoid, and exactly the tools and resources you need — nothing more.

Principle	What it means	Why it matters
Phase by phase	Do not skip or reorder phases	Each phase builds on the previous one — gaps compound
Tools on demand	Buy only what the current phase needs	Avoids wasting money on gear you won't touch for months
Everything legal	Only test devices you own or have written permission to test	Keeps your practice 100% ethical and employable
Document always	Commit notes, scripts, and write-ups to GitHub	Your portfolio is built one commit at a time
Setup help in chat	For install steps, ask Claude when you reach that step	Exact steps change across versions; live guidance is more accurate

✔ START HERE

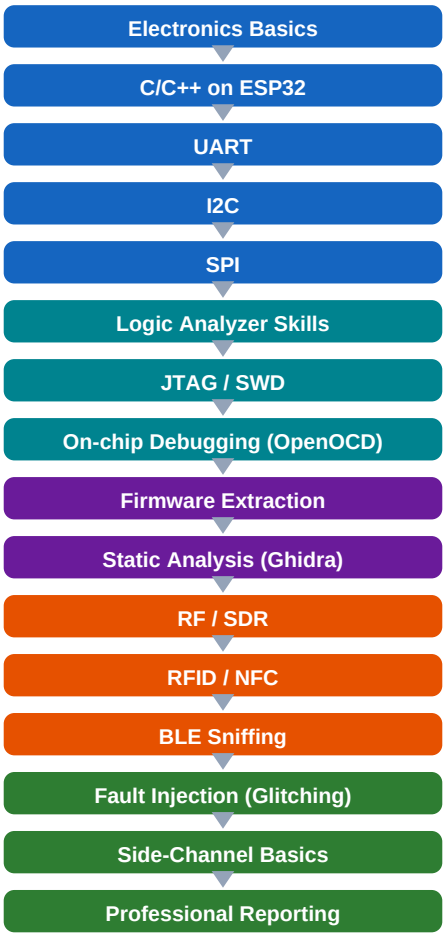
Day 1 action: create a GitHub repository called **embedded-security-journey** right now. Every command you run, every note you take, every project you complete goes in there. In 8 months that repo is your CV.

■ LEGAL REMINDER

Important: Only test equipment you personally own or have explicit, written authorisation to test. This is both a legal requirement and a professional standard in the security industry.

Your Skill Tree

This is the exact order skills stack on top of each other. Each box depends on the one above it — do not jump ahead. Use this page as your mental map of the whole journey.



Color	Phase
	Phase 1 — Foundations
	Phase 2 — Debug Interfaces
	Phase 3 — Firmware & RE
	Phase 4 — RF & Wireless
	Phase 5 — Advanced / Pro

Roadmap Overview

Phase	Weeks	Core Skill Gained	Starter Tools	End Goal
1 — Foundations	1–6	Electronics, C/C++, serial protocols	ESP32, multimeter, USB-UART, logic analyzer	Read raw I2C/UART traffic from a real device
2 — Debug Interfaces	6–12	JTAG/SWD, on-chip debugging	ST-Link V2	Attach a debugger to an unknown board
3 — Firmware & RE	12–20	Firmware extraction, Ghidra, binwalk, Python scripting	CH341A + SOIC8 clip	Extract & analyse a real device's firmware
4 — RF & Wireless	20–30	SDR, RFID, BLE sniffing, WiFi security	RTL-SDR, RC522	Capture, decode, and clone a wireless signal
5 — Advanced / Pro	30–42+	Fault injection, side-channel, CTFs	DIY glitch rig, CTF platforms	Public write-ups + job-ready portfolio

Total estimated time: **9–12 months** at 6–8 focused hours per week. Firmware/RE (Phase 3) commonly runs longer than its nominal 8 weeks once you're working with real, symbol-less firmware — budget for that rather than compressing it. updated

Starter Budget Breakdown (Phase 1 only)

Item	Approx. EGP	Where (Egypt)
ESP32 Dev Board	200 – 350	RAM Electronics · Maamoon · Amazon.eg
Multimeter (basic)	150 – 300	Ataba electronics market · Amazon.eg
USB-to-UART (CP2102/FTDI)	40 – 80	RAM Electronics · local electronics market
Soldering iron kit	300 – 600	Ataba market · RAM Electronics
Breadboard + jumper wires	80 – 150	RAM Electronics · Amazon.eg
USB Logic Analyzer (8ch) moved from Phase 2	200 – 400	Amazon.eg · AliExpress (cheaper)
Phase 1 Total	**970 – 1,880 EGP**	

Search every item on eg.pricena.com before buying — it compares Amazon.eg and local stores in one page. The logic analyzer moved here from Phase 2 because Phase 1's own sensor project asks you to use it.

1 Foundations

Weeks 1–6

Very Easy Phase 1 of 5

■ PHASE GOAL

Be comfortable wiring circuits, reading datasheets, writing basic C/C++ firmware, and communicating with a microcontroller over UART, I2C, and SPI.

What to Learn

- Electronics basics: voltage, current, resistance, Ohm's law, pull-up/pull-down resistors, continuity testing with a multimeter.
- Microcontroller programming in C/C++ with the ESP32 — start with Arduino IDE, then explore ESP-IDF for deeper control.
- Serial protocols at the signal level (not just library calls): UART framing, I2C addresses, SPI clock/data modes.
- How to read a datasheet and a PCB schematic: find pinouts, identify test points, check voltage levels (3.3V vs 5V).

Common Mistakes to Avoid

- Wiring a 5V device directly to a 3.3V ESP32 pin — use a level shifter.
- Guessing the baud rate instead of identifying it (start with 115200, then 9600).
- Skipping the datasheet — always find the official datasheet before wiring anything.
- Not committing progress to GitHub from day one — it looks bad to start later.

Week-by-Week Plan

Week 1
Install Arduino IDE, set up ESP32 board support, blink an LED

Week 2
Wire a DHT22 sensor — a single-wire digital protocol, not I2C fixed — read data in Serial Monitor. Want real I2C practice instead? Swap in a BME280/AHT10 module — see Appendix E.

Week 3
Use USB-UART adapter to read ESP32 boot log at correct baud rate

Week 4
Wire SPI device (e.g. SD card module) — read/write a file

Week 5
Buy an old router — find and attach to UART port, capture boot log

Week 6
Solder header pins onto a spare board, then desolder them cleanly — practice before Phase 3 needs it added. Then document everything in GitHub — write your first README.

Hands-on Projects

PROJECT 1

ESP32 Sensor Node [1/5 · Very Easy]

Wire a DHT22 temperature/humidity sensor (single-wire protocol — see note above for a true-I2C alternative). View the raw signal timing on your logic analyzer, not just the library output. fixed Confirm the byte sequence matches the datasheet.

PROJECT 2

Router UART Boot Log [1/5 · Very Easy]

Buy a cheap used router (TP-Link / D-Link). Find its UART header pins using a multimeter, set the correct baud rate, connect your USB-UART adapter, and capture the full boot log to a file.

Corrections applied on this page vs. the original draft: DHT22's protocol description (it is not I2C) and the logic analyzer's phase placement. Full reasoning in Appendix E.

Quick Setup Steps

- Download Arduino IDE from the official Arduino website (search: 'Arduino IDE download').
- In Preferences, add the Espressif ESP32 boards URL (search: 'install ESP32 board Arduino IDE 2026' for the current URL).
- Open Boards Manager, search 'esp32', install the package by Espressif Systems.
- Plug in your ESP32 via USB. In Tools, select the right Board and Port, then upload the built-in Blink example.
- If the port is not detected: install the CP2102 or CH340 USB driver (search chip name + 'driver' + your OS version).

Tools This Phase

Tool	Approx. EGP	Where in Egypt
ESP32 Dev Board	200–350	RAM Electronics · Maamoon · Amazon.eg
Multimeter (basic)	150–300	Ataba electronics market · Amazon.eg
USB-to-UART (CP2102/FTDI)	40–80	RAM Electronics · local electronics market
Soldering iron kit	300–600	Ataba market · RAM Electronics
Breadboard + jumper wires	80–150	RAM Electronics · Amazon.eg
USB Logic Analyzer (8ch) moved here	200–400	Amazon.eg · AliExpress (cheaper)

If you're stuck — common errors & fixes	
ESP32 not detected by IDE	→ Install CP2102/CH340 driver, try a different USB cable (must support data, not charge-only)
Upload fails / times out	→ Hold the BOOT button while upload starts, release once it begins
Serial Monitor shows garbage text	→ Wrong baud rate — match the Serial.begin() value in your code
DHT22 gives NaN / no reading	→ Confirm the DATA pin has a pull-up resistor (often built into the breakout board already) — this is the most common single point of failure, not a bad sensor added

Resources

- Book: Practical IoT Hacking — Chantzis et al. (2020) — Chapters 1–3 cover exactly this foundation.
- GitHub: github.com/fkie-cad/awesome-embedded-and-iot-security — star this now, beginner case studies section.
- Tool: Arduino IDE — arduino.cc/en/software (free, Windows/Mac/Linux)
- Tool: ESP-IDF Docs — docs.espressif.com (reference once you outgrow Arduino IDE)
- **New:** Adafruit "DHT sensor library" (Arduino Library Manager) — needed for Project 1 regardless of which sensor variant you use added

2 Debug Interfaces: JTAG / SWD / UART

Weeks 6–12

●●○○○ Easy

Phase 2 of 5

■ PHASE GOAL

Given an unknown board, find its debug pins using a multimeter and logic analyzer, then attach a debugger and halt the CPU.

What to Learn

• JTAG fundamentals: TAP state machine, boundary scan, why manufacturers leave debug headers on production boards.
• SWD (Serial Wire Debug): the ARM-specific two-wire alternative to JTAG — faster to identify and attach.
• Pin identification workflow: visual inspection → continuity testing → signal probing with logic analyzer → brute-force enumeration.
• Using OpenOCD (free, open-source) with an ST-Link clone to halt, read memory, and dump firmware over JTAG/SWD.

Common Mistakes to Avoid

• Touching SWD/JTAG pins on a powered board with metal probes — power down first or use insulated hooks.
• Wrong voltage on debug lines — always check if the target runs at 3.3V or 1.8V before connecting.
• Not installing correct OpenOCD interface config — the wrong config for your ST-Link version gives silent errors.
• Giving up on pin identification after 10 minutes — it takes patience, be systematic.

Week-by-Week Plan

Week 6–7
Study JTAG/SWD theory — read the ARM debug interface spec overview

Week 7–8
Connect ST-Link to an STM32 Blue Pill via SWD — halt CPU in OpenOCD

Week 8–9
Use logic analyzer to identify UART pins on an unmarked dev board

Week 9–10
Practice pin identification on a second board without the datasheet

Week 10–11
Dump firmware from STM32 via OpenOCD — verify it against known content

Week 11–12
Write a full report: board, pins found, method, findings — push to GitHub

Hands-on Projects

PROJECT 1

STM32 SWD Debug [2/5 · Easy]

Take an STM32 'Blue Pill' (cheap, common). Connect via SWD using ST-Link clone + OpenOCD. Halt the CPU, read the memory map, and confirm you can read and write registers. Document every command.

PROJECT 2

Unlabelled Board Pin Hunt [2/5 · Easy]

Take any board without markings. Use only a multimeter and logic analyzer to find UART TX/RX pins. Connect and capture whatever the board outputs. Write up your methodology.

Tools This Phase

Tool	Approx. EGP	Where in Egypt

ST-Link V2 clone	150–250	RAM Electronics · Amazon.eg
USB Logic Analyzer (8ch 24MHz)	200–400	Amazon.eg · AliExpress (cheaper, longer shipping)
STM32 Blue Pill board (target)	80–150	RAM Electronics · Amazon.eg
Test hooks / fine probe clips	50–120	Electronics market · Amazon.eg

If you're stuck — common errors & fixes	
OpenOCD can't find target	→ Wrong interface/target config file — match exact ST-Link version and MCU
No response on SWD pins	→ Check 3.3V power to target board, confirm SWDIO/SWCLK not swapped
Logic analyzer shows no signal	→ Ground (GND) not connected between analyzer and target — always share ground
Can't identify UART TX pin	→ TX idles HIGH at rest and dips briefly on boot messages — probe during power-on

Resources

- Tool: OpenOCD — openocd.org (free JTAG/SWD debugger, works with ST-Link and J-Link clones)
- GitHub: github.com/hexsecs/awesome-embedded-security — tools section: JTAGenum, UrJTAG, Hydrabus
- Talk: 'Intro to Hardware Hacking' — Philippe Laulheret, DEF CON 27 (search title on YouTube)
- Book: The Hardware Hacking Handbook — van Woudenberg & O'Flynn — start reading from Chapter 1

3 Firmware Extraction & Reverse Engineering

Weeks 12-20

●●○○○ Medium Phase 3 of 5

■ PHASE GOAL

Pull firmware off a real device, extract its filesystem with binwalk, and find at least one real vulnerability using static analysis in Ghidra.

⚠ **Standing rule for this entire phase:** whenever a tool here has a Python API (binwalk, Ghidra's scripting console, and others you'll meet later in Phase 4) spend 30–60 minutes scripting it instead of using the GUI alone before moving to the next task. This is where Python becomes a real skill rather than a separate "learn Python" week — by Phase 5 it compounds into genuine competency. added

What to Learn

- Firmware extraction methods: JTAG/SWD dump, UART shell, SPI flash in-circuit reading with CH341A, vendor update files.
- Using binwalk to identify file formats, compression schemes, and extract embedded file systems automatically.
- Static analysis in Ghidra: loading firmware, identifying functions, finding strings, tracing code paths.
- Common embedded vulnerability classes: hardcoded credentials, debug services left open, insecure update parsers, buffer overflows in input handling.

Common Mistakes to Avoid

- Reading SPI flash at 3.3V when the chip expects 1.8V — destroys the chip. Check the datasheet voltage first.
- **Trusting your CH341A's data lines to actually be 3.3V.** Most cheap CH341A boards run SPI data lines at 5V regardless of the target-voltage jumper, because the CH341 chip itself is powered from USB 5V. This can destroy a 3.3V or 1.8V flash chip. Measure MISO/MOSI with your multimeter before your first dump — search "CH341A 3.3V fix" if you read close to 5V. critical
- Confusing chip package types (SOIC8 vs WSON8) — buy both clip types before you sit down to work.
- Only running binwalk -e — also run with -M (matryoshka) to recurse into nested archives.
- Skipping Ghidra because 'it's too hard' — spend one day on the official tutorial and it clicks fast.

Week-by-Week Plan

Week 12–13
Install binwalk — run against 3 public firmware update .bin files

Week 13–14
Install Ghidra — load a simple ARM ELF binary and find main()

Week 14–15
Set up IoTGoat VM — find all 10 intentional vulnerabilities

Week 15–16
Multimeter-check your CH341A's data-line voltage first added , then wire CH341A + SOIC8 clip to an SPI flash chip — do a test dump

Week 16–17
Dump SPI flash from an old router — compare against public firmware

Week 17–18
Load extracted binary into Ghidra — trace one function end to end

Week 18–20
Write a full analysis report with CVE-style severity — push to GitHub

Hands-on Projects

PROJECT 1

Public Firmware Analysis [3/5 · Medium]

Download 3 real firmware update files from manufacturers' support pages. Run binwalk -e on each. Document the filesystem structure, find any hardcoded strings, config files, or credentials in each one.

PROJECT 2

IoTGoat Complete Walkthrough [3/5 · Medium]

Set up the IoTGoat deliberately-vulnerable firmware (github.com/OWASP/IoTGoat). Find and document all 10 intentional vulnerabilities. Write a structured report for each: location, impact, remediation.

PROJECT 3

SPI Flash Dump [3/5 · Medium]

Identify the SPI flash chip on an old router or IoT device you own. **Before clipping on, measure your CH341A's MISO/MOSI voltage with a multimeter** — most cheap units output 5V regardless of the target jumper and will damage a 3.3V/1.8V chip. critical Apply the "CH341A 3.3V fix" if needed. Then use the SOIC8 clip for an in-circuit read. Verify the dump is valid with binwalk, then compare to the public firmware.

Tools This Phase

Tool	Approx. EGP	Where in Egypt
CH341A USB programmer	100–200	AliExpress · check RAM Electronics for stock
SOIC8 / WSON8 test clip	50–150	AliExpress · local electronics market
Bus Pirate (optional upgrade)	400–900	AliExpress / import — rare locally

If you're stuck — common errors & fixes	
binwalk -e extracts nothing	→ Try binwalk -Me (matryoshka) to recurse into nested/compressed archives
SPI clip reads all 0xFF or 0x00	→ Clip not seated correctly on chip pin 1 — check orientation dot on chip
Ghidra import fails / wrong architecture	→ Manually set processor type (ARM/MIPS) — check chip markings for CPU family
Flash chip reads garbage or dies after first dump	→ Almost always the CH341A 5V data-line issue — verify with a multimeter before your next attempt, apply the 3.3V fix added

Resources

- GitHub: github.com/ReFirmLabs/binwalk — firmware extraction tool (must install)
- GitHub: github.com/NationalSecurityAgency/ghidra — free NSA reverse engineering suite
- GitHub: github.com/OWASP/IoTGoat — deliberately vulnerable firmware for practice
- GitHub: github.com/fkie-cad/FACT_core — Firmware Analysis & Comparison Tool (automates much of Phase 3)
- Book: The IoT Hacker's Handbook — Aditya Gupta (2019) — firmware analysis workflow chapters
- Search "CH341A 3.3V fix" before your first flash dump — a well-documented, reversible hardware mod added

4 RF & Wireless Security

Weeks 20-30

●●●○○ Medium Phase 4 of 5

■ PHASE GOAL

Capture an unknown sub-GHz RF signal, identify its modulation, decode its data, read/clone an RFID card you own, and capture/analyze a WiFi handshake on your own network. expanded

What to Learn

- Software Defined Radio (SDR) basics: sampling, IQ data, AM/FM/ASK/FSK/OOK modulation recognition.
- Sub-GHz analysis (433/315 MHz): capturing remote signals, identifying encoding (Manchester, NRZ), replay attacks.
- RFID/NFC: 125kHz (EM4100, HID) and 13.56MHz (MIFARE) — how access control systems work and where they fail.
- Bluetooth Low Energy: advertisement sniffing, GATT service enumeration, identifying unauthenticated characteristics.
- **WiFi security basics:** WPA2 4-way handshake capture, and why offline cracking targets the handshake, not the live connection. new

Common Mistakes to Avoid

- Only capturing one packet — capture 5–10 repeats to confirm the encoding is consistent.
- Trying to clone 13.56MHz MIFARE Classic without first checking if it has anti-cloning protections.
- Forgetting antenna placement — hold the RTL-SDR antenna perpendicular to the floor for best reception.
- Analysing someone else's device/signal/network without permission — always use equipment and networks you personally own.

Week-by-Week Plan

Week 20–21
Install SDR# / GQRX — receive FM radio to confirm RTL-SDR works

Week 21–22
Capture a 433MHz remote signal in Universal Radio Hacker (URH)

Week 22–23
Decode the remote's encoding — identify bit pattern and timing

Week 23–24
Wire RC522 to ESP32 — read UID from an RFID card you own

Week 24–25
Clone a 125kHz card onto a writable T5577 blank card

Week 25–26
Sniff BLE advertisements from a smart device you own

Week 26–27
Capture a WPA2 handshake on your own WiFi with aircrack-ng, analyze in Wireshark new

Week 27–30
Write up all four attacks with screenshots — push to GitHub extended

Hands-on Projects

PROJECT 1

433MHz Remote Analysis [3/5 · Medium]

Use your RTL-SDR and Universal Radio Hacker to capture the signal from a wireless remote you own (doorbell, fan remote, etc.). Identify the modulation, decode the bit pattern, and document the protocol completely.

PROJECT 2

RFID Card Read & Clone [3/5 · Medium]

Use an RC522 module wired to your ESP32 to read the UID and data blocks from an RFID card you own. Then attempt to clone it to a writable T5577 blank card. Document which security measures (if any) block you.

PROJECT 3

BLE Device Enumeration [3/5 · Medium]

Sniff BLE advertisement packets from a cheap smart bulb or fitness band you own. Use an ESP32 or nRF52840 dongle to enumerate its GATT services. Look for characteristics that accept writes without authentication.

PROJECT 4 new

WiFi Handshake Capture & Analysis [3/5 · Medium]

On your own home WiFi network, use the aircrack-ng suite to capture a WPA2 4-way handshake when a device joins (or briefly deauthenticates and reconnects). Open the capture in Wireshark and identify each of the 4 EAPOL frames. Document which part of the handshake is what offline cracking tools actually target, and why the live connection itself isn't what's "cracked."

Tools This Phase

Tool	Approx. EGP	Where in Egypt
RTL-SDR dongle	450–700	Amazon.eg
RC522 RFID module + test cards	100–150	RAM Electronics
T5577 writable RFID cards (pack)	80–150	AliExpress
nRF52840 dongle (BLE, optional)	350–600	AliExpress / import
aircrack-ng suite new — free	0 (software)	Already covered in Appendix A — no new purchase

If you're stuck — common errors & fixes

RTL-SDR shows only noise	→ Check antenna is attached and extended, try a known strong FM station first
RC522 read fails intermittently	→ Check 3.3V (not 5V!) power — RC522 is 3.3V only and can be damaged by 5V
BLE scanner shows no devices	→ Confirm target device advertising is active — many devices only advertise briefly after power-on
No WiFi handshake captured	→ A brief deauth forces a connected device to reconnect and re-handshake — only ever on a network you own new

Resources

- GitHub: github.com/jopohl/urh — Universal Radio Hacker (best tool for unknown RF protocol analysis)
- GitHub: github.com/merbanan/rtl_433 — decodes hundreds of known 433/868MHz device protocols out of the box
- Book: Inside Radio: An Attack and Defense Guide — Yang & Huang — sub-GHz and car-key security depth
- GitHub: github.com/aircrack-ng/aircrack-ng — the WiFi auditing suite for Project 4 new

5 Advanced & Professional

Weeks 28–40+

●●●●● Expert

Phase 5 of 5

■ PHASE GOAL

Operate like a professional hardware security researcher: fault injection basics, CTF wins, published write-ups, and a GitHub portfolio a client or employer can review.

What to Learn

• Voltage glitching: how a brief power-line disturbance can cause a CPU to skip an instruction — used to bypass secure boot or readout protection on weak MCUs.
• Introduction to power side-channel analysis: what power traces reveal about what code is executing and which key bytes are being processed.
• Professional reporting: reproduction steps, root-cause analysis, impact assessment, CVSS-style severity, and actionable remediation.
• Responsible disclosure process: how to contact vendors, what to include, timeline expectations, and when/how to publish.

Common Mistakes to Avoid

• Skipping CTFs because 'they're not real work' — CTF write-ups are the fastest way to build a visible portfolio.
• Publishing a write-up about a vulnerability in a device before attempting responsible disclosure to the vendor.
• Buying a Flipper Zero or ChipWhisperer before mastering Phases 1–4 — expensive tools do not compensate for gaps in fundamentals.
• Treating the portfolio as an afterthought — employers and clients read your GitHub before they read your CV.

Week-by-Week Plan

Week 28–30
Complete 5 Microcorruption challenges — document each solution

Week 30–32
Solve 2–3 Riscure Rhme CTF challenges — write a public write-up

Week 32–34
Study voltage glitching theory — build a basic DIY glitch trigger circuit

Week 34–36
Attempt a documented STM32 RDP bypass via glitching on your own board

Week 36–38
Pick one device you own — do a full mini-pentest — write a pro report

Week 38–40
Polish GitHub profile, add READMEs to all projects, write a bio

Hands-on Projects

PROJECT 1

CTF Portfolio [5/5 · Expert]

Complete at least 5 Microcorruption challenges and 3 Riscure Rhme challenges. For each one, write a structured write-up: challenge description, your approach, the vulnerability, and the solution. Publish all of these on GitHub.

PROJECT 2

Full Device Assessment [5/5 · Expert]

Pick one IoT device you own (smart plug, IP camera, etc.). Conduct a complete assessment: recon, UART/JTAG identification, firmware extraction and analysis, wireless protocol analysis. Write a professional pentest report and publish it.

PROJECT 3

Voltage Glitch Experiment [5/5 · Expert]

Follow a documented STM32 readout-protection bypass tutorial using voltage glitching on your own Blue Pill board. Build a simple glitch trigger circuit. Document your success rate, timing parameters, and observations.

Tools This Phase

Tool	Approx. EGP	Where in Egypt
ChipWhisperer Lite (optional, best glitch tool)	~8,000–12,000	NewAE store / import only
DIY glitch rig (MOSFETs + resistors — budget alternative)	100–250	Local electronics market
Raspberry Pi Pico (cheap glitch controller alternative)	200–350	Amazon.eg

If you're stuck — common errors & fixes	
Glitch never triggers a fault	→ Sweep glitch timing/width systematically — start wide range then narrow down
Target resets instead of glitching	→ Glitch pulse too strong/long — reduce width incrementally
CTF challenge feels impossible	→ Re-read the challenge brief for hints — most embedded CTFs reward patience over cleverness
Report feels unprofessional	→ Follow a public disclosure template (search 'vulnerability report template github')

Resources

- CTF: microcorruption.com — free browser-based embedded CTF, excellent for building assembly debugging skills
- CTF: Riscure Rhme series (2015–2018) — search 'Riscure rhme github' for challenges and community write-ups
- GitHub: github.com/IamAlch3mist/Awesome-Embedded-Systems-Vulnerability-Research — real CVE write-ups and talks
- Book: The Hardware Hacking Handbook — van Woudenberg & O'Flynn — read it cover to cover by this phase
- Conferences (YouTube): DEF CON IoT Village, Hardwear.io, CCC (Chaos Communication Congress)

Complete Tool & Budget Reference

Buy tools incrementally, phase by phase. Prices are approximate EGP as of mid-2026 — always check **eg.pricena.com** before buying to compare Amazon.eg and local stores.

Tool	Phase	Approx. EGP	Where (Egypt)	Priority
ESP32 Dev Board	1	200–350	RAM Electronics · Maamoon · Amazon.eg	**Essential**
Multimeter	1	150–300	Ataba market · Amazon.eg	**Essential**
USB-to-UART (CP2102/FTDI)	1	40–80	RAM Electronics · local market	**Essential**
Soldering iron kit	1	300–600	Ataba market · RAM Electronics	**Essential**
Breadboard + jumper wires	1	80–150	RAM Electronics · Amazon.eg	**Essential**
USB Logic Analyzer (8ch) moved from Phase 2	1	200–400	Amazon.eg · AliExpress (cheaper)	**Essential**
ST-Link V2 clone	2	150–250	RAM Electronics · Amazon.eg	**Essential**
STM32 Blue Pill (target)	2	80–150	RAM Electronics · Amazon.eg	**Essential**
CH341A programmer + SOIC8 clip	3	150–300	AliExpress mainly · check RAM Electronics	**Essential**
Bus Pirate / Hydrabus	3	400–900	AliExpress / import (rare locally)	**Optional**
RTL-SDR dongle	4	450–700	Amazon.eg	**Essential**
RC522 RFID module + cards	4	100–150	RAM Electronics	**Essential**
T5577 writable RFID cards	4	80–150	AliExpress	**Essential**
nRF52840 USB dongle (BLE)	4	350–600	AliExpress / import	**Optional**
DIY glitch rig (MOSFET kit)	5	100–250	Local electronics market	**Optional**
Raspberry Pi Pico (glitch ctrl)	5	200–350	Amazon.eg	**Optional**
Used oscilloscope (2ch, basic) new	5	1,500–4,000	Used/2nd-hand electronics listings	**Optional**
ChipWhisperer Lite	5	8,000–12,000	NewAE.com / import only	**Advanced**
Flipper Zero	5+	~20,000+	Import only — not worth buying before Phase 4	**Advanced**

A basic used oscilloscope is added here (Phase 5, Optional) — without one, the side-channel section of Phase 5 stays conceptual only, since power traces can't actually be viewed without a scope. **new note**

GitHub Reference Library

Star all of these repositories now. You will return to them constantly throughout every phase.

Master Curated Lists — Start Here

- github.com/fkie-cad/awesome-embedded-and-iot-security — the definitive list: tools, books, CTFs, case studies. Star this first.
- github.com/hexsecs/awesome-embedded-security — strong on fuzzing, hardware debug tools, and academic research.
- github.com/kayranfatih/awesome-iot-and-hardware-security — excellent hardware tool catalogue (Tigard, Glasgow, GreatFET).
- github.com/IamAlch3mist/Awesome-Embedded-Systems-Vulnerability-Research — real CVE write-ups and conference talks.
- github.com/CyberSecurityUP/Awesome-Hardware-and-IoT-Hacking — also covers automotive/CAN-bus.

Core Software Tools

- github.com/ReFirmLabs/binwalk — firmware extraction and analysis (Phase 3)
- github.com/NationalSecurityAgency/ghidra — free NSA reverse engineering suite (Phase 3)
- github.com/openocd-org/openocd — on-chip debugger for JTAG/SWD (Phase 2)
- github.com/jopohl/urh — Universal Radio Hacker, unknown RF protocol analysis (Phase 4)
- github.com/merbanan/rtl_433 — decodes hundreds of known sub-GHz device protocols (Phase 4)
- github.com/fkie-cad/FACT_core — automated firmware analysis and comparison (Phase 3)

Practice Targets — Legal & Safe

- github.com/OWASP/IoTGoat — deliberately insecure OpenWrt firmware — 10 documented vulnerabilities to find
- microcorruption.com — free browser-based embedded CTF — excellent assembly and debugging practice
- Search 'Riscure rhme github' — real hardware CTF challenges from 2015–2018 with community write-ups

Open Hardware Tool Designs

- JTAGulator — JTAG/UART pin identification tool; schematics on GitHub or buy a clone
- Tigard — FT2232H-based multi-protocol hacking tool; open hardware design, easy to build or buy
- Glasgow Interface Explorer — FPGA-based multitool for SPI/I2C/UART/JTAG and custom protocols

> **✔ YOUR PORTFOLIO REPO**
>
> Create a GitHub repo named **embedded-security-journey** today. Commit every command, note, script, and project write-up as you go. In 8–10 months, that repo is both your learning record and your CV. Employers in this field read GitHub profiles before they read resumes.

Reference Books

Listed in the order you should read them — earlier phases first.

Book	Author(s)	Read in	Best for
Practical IoT Hacking	Chantzis, Deirmentzoglou, Stais, Calderon, Woods (2020)	Phase 1–2	Broad practical foundation — hardware, firmware, and wireless in one book
The Hardware Hacking Handbook	Jasper van Woudenberg, Colin O'Flynn (2020)	Phase 2–5	The field's core reference — JTAG, glitching, side-channel; read cover to cover
The IoT Hacker's Handbook	Aditya Gupta (2019)	Phase 3	Firmware extraction and analysis workflow, very hands-on
Inside Radio: An Attack and Defense Guide	Qing Yang, Lin Huang	Phase 4	Deep coverage of sub-GHz, car keys, RFID/NFC attack techniques
The Hacker's Hardware Toolkit	Yago Hansen (2019)	All phases	Tool selection and hardware hacking methodology reference

Capstone Project — Full Device Assessment

This is the project that ties every phase together. Save it for after you complete Phase 4 (or do it alongside Phase 5). Pick **one** IoT device you own — an old router, a smart plug, an IP camera, or similar — and take it through a complete, professional assessment from start to finish.

Step 1: Recon
Identify chipset, board revision, FCC ID if available, and search for any public information or known issues.

Step 2: Physical Access
Open the device, identify UART/JTAG/SWD headers using Phase 1–2 skills.

Step 3: Firmware Extraction
Dump firmware via UART shell, SPI flash read, or vendor update file (Phase 3 skills).

Step 4: Static Analysis
Run binwalk + Ghidra on the extracted firmware. Look for hardcoded credentials, open services, weak update logic.

Step 5: Wireless Assessment
If the device has WiFi/BLE/RF, capture and analyze its wireless behavior (Phase 4 skills).

Step 6: Report Writing
Write a full professional report: executive summary, findings, severity ratings, and remediation advice.

Step 7: Publish
Push the full write-up (redacting anything sensitive) to your GitHub portfolio with photos and code.

✔ WHY THIS MATTERS

This single project, done well and documented publicly, is worth more to your career than any certificate. It proves you can do the full job, not just isolated exercises.

Budget vs. Professional Tools

Every tool in this roadmap has a professional-grade equivalent used by full-time researchers. You do not need these to learn — the budget tools teach the same fundamentals. Use this table to know what to upgrade to once you are earning from these skills or working professionally.

Purpose	Budget Tool (this roadmap)	Professional Upgrade	Why upgrade eventually
SPI/Flash programming	CH341A	Bus Pirate / Hydrabus	Faster, supports more protocols, better software support
SDR	RTL-SDR	HackRF One	Transmit capability, wider frequency range
JTAG/SWD debugging	ST-Link clone	Segger J-Link	Much faster flashing/debugging, official ARM support
Target dev board	ESP32 / STM32 Blue Pill	STM32 Nucleo / Discovery	Better documentation, on-board debugger included
RFID/NFC	RC522	Proxmark3	Supports advanced cloning and protocol attacks RC522 cannot do
Fault injection	DIY glitch rig	ChipWhisperer	Precise timing control, built-in analysis software

Rule of thumb: upgrade a tool only after you have hit its actual limitation in a real project — not before.

Project Write-up Template

Use this exact structure for every project you publish on GitHub. Consistency makes your portfolio look professional and makes it easy for a reviewer to scan quickly.

Section	What goes here
Project Name	One clear line — what device/target and what you did to it.
Objective	What you set out to find or achieve. One or two sentences.
Hardware Used	Exact tools and boards, including model numbers.
Method	Step-by-step what you actually did, in order.
Photos / Screenshots	Visual proof — wiring setup, terminal output, tool screenshots.
Findings	What you discovered. Be specific — exact strings, addresses, protocols.
Commands / Code	Exact commands or scripts used, so someone else can reproduce your work.
Lessons Learned	What you would do differently next time — this shows growth, not just results.

> **TIP**
>
> Save this as a file called **TEMPLATE.md** in your **embedded-security-journey** repo. Copy it into a new folder for every project so the structure is always ready.

Career Progression

This is the typical route people take in this field. You do not need to hit every rung — many hardware hackers stay at 'Firmware Engineer' or 'IoT Security Engineer' level happily, and freelance/bug-bounty work is a valid path at any stage past Phase 3.

Stage	Typical Focus	Roughly maps to
1. Junior Embedded Engineer	Firmware development, basic hardware bring-up	End of Phase 1–2
2. Firmware Engineer	Embedded C/C++, driver development, debugging with JTAG/SWD	End of Phase 2–3
3. IoT Security Engineer	Firmware security review, secure boot, threat modelling	End of Phase 3
4. Hardware Penetration Tester	Full device assessments, client engagements, reporting	End of Phase 4
5. Red Team / Researcher	Novel vulnerability research, fault injection, conference talks	Phase 5 and ongoing

Note on salaries: reliable, specific salary data for this niche specialization — especially broken down by country — is not something that can be stated accurately here. Ranges vary hugely by company, remote vs local, and experience. When you are ready to job-search, check current listings on LinkedIn and Wuzzuf for Egypt-based roles, and levels.fyi / Glassdoor for international/remote roles, rather than relying on any fixed number in this document.

Stay Connected — Community & Ongoing Learning

Hardware hacking is a small, active community. Being present in it — asking questions, reading write-ups, watching talks — accelerates learning far more than solo study.

Where to engage

- r/AskElectronics and r/ReverseEngineering on Reddit — active, beginner-friendly, good for troubleshooting.
- DEF CON and Hardwear.io conference talks on YouTube — free, and cover cutting-edge real-world research.
- Search 'hardware hacking discord' and 'ICS/OT security discord' for current active servers — these communities move between servers over time, so search for the current one rather than relying on a fixed invite link.
- Follow hardware security researchers on X/Twitter and Mastodon — many post write-ups and CTF hints in real time.

Ongoing CTFs and competitions

- DEF CON CTF Qualifiers (yearly) — has hardware-adjacent challenges alongside software ones.
- Hardwear.io CTF — hardware-specific, runs alongside their annual conference.
- Search 'hardware CTF calendar' periodically — new embedded/IoT CTFs appear throughout the year.

Communities and server links change over time — when you reach this stage, ask Claude to search for current active Discord servers and communities so you get live, working links instead of dead ones.

Milestones — How You'll Know You're on Track

After Phase	You can do this without help
Phase 1	Wire an ESP32, write C firmware, read raw I2C/UART traffic on a logic analyzer, and identify UART pins on an unknown board.
Phase 2	Given any unlabelled board, find its debug pins within an hour, attach OpenOCD, halt the CPU, and read a memory region.
Phase 3	Extract firmware from a real device, run binwalk, open the result in Ghidra, and find at least one real issue (hardcoded creds, open debug port, etc.).
Phase 4	Capture and decode an unknown sub-GHz signal, read an RFID card you own, and enumerate BLE GATT services on a smart device.
Phase 5	Publish 3+ CTF write-ups, complete one full IoT device pentest report, and maintain a GitHub portfolio a client or employer can review today.

FINAL NOTE

This field rewards patience and hands-on repetition more than raw talent. Document every step — your future self and your future clients will thank you. Good luck.

APPENDIX A

The Mr. Robot Toolkit Reference

Real tools from the show, mapped to your Hardware Hacking & Embedded Security Roadmap — season by season, phase by phase.

A Why This Appendix Exists

Mr. Robot is famous among security professionals for using **real tools, correctly**, instead of invented Hollywood software. Its technical consultants (including Ryan Kazanciyan and Marc Rogers) made sure almost every hack on screen mirrors an actual technique. That makes the show a genuinely useful map of the industry's toolset — this appendix connects each tool the characters use to where it fits in your existing 5-phase roadmap, so you're not learning tools at random.

Legal reminder: every tool below is legitimate security research / penetration testing software used daily by professionals. Only ever point them at devices and networks you own or have written authorization to test — exactly the same rule as the rest of your roadmap.

How to Read the Phase Tags

Phase 1 Foundations Phase 2 Debug Interfaces Phase 3 Firmware & RE Phase 4 RF & Wireless Phase 5 Advanced / Pro
Outside Software/network security, adjacent to this roadmap

B Software Tools Used On Screen

Operating System & Exploitation Frameworks

Tool	What it does	Where it appears	Roadmap link
Kali Linux	Debian-based OS pre-loaded with hundreds of pentest tools	Elliot & fsociety's default OS throughout the series	Outside — good general VM to run alongside your roadmap
Metasploit	Exploit development & delivery framework	Used with BlueSniff/Bluetooth tools, S1E6	Outside — network pentesting, not hardware
Social-Engineer Toolkit (SET)	Automates phishing/SMS-spoofing style attacks	Multiple social-engineering scenes	Outside
Shodan	Search engine for internet-connected devices/open ports	S3E1 — Elliot searches for E Corp's exposed Apache Tomcat servers	Outside — great for recon step of your Capstone Project

Password & Credential Tools

Tool	What it does	Where it appears	Roadmap link
John the Ripper	Offline password hash cracker	S1 — cracking a password hash after a Shellshock exploit	Outside
Hashcat	GPU-accelerated hash cracking	S3 finale — cracking Romero's LUKS-encrypted keylogger volume	Outside
CUPP / CeWL	Builds targeted wordlists from OSINT (names, dates, interests)	Used before the hash-cracking scenes above	Outside

Network & Traffic Analysis

Tool	What it does	Where it appears	Roadmap link
Wireshark	Packet capture & protocol analysis	Multiple episodes, incl. Tor exit-node traffic capture	Phase 2 useful once you're staring at UART/JTAG traffic patterns too
tcpdump	Command-line packet capture	Same Tor exit-node scenes	Outside
Nmap	Network/port scanner	Background use across the series	Outside

Data Hiding & Encryption

Tool	What it does	Where it appears	Roadmap link
ProtonMail	Zero-access encrypted email, Switzerland-based	Elliot & fsociety's email throughout	Outside — good habit for your own OPSEC while researching
DeepSound	Steganography — hides files inside audio	S1E8 — Elliot hides files on a floppy disk among music CDs	Outside
LUKS (cryptsetup)	Linux disk encryption	S3/S4 — Romero's keylogger storage, and Elliot & Darlene's own drives	Phase 3 relevant once you're thinking about how vendors protect firmware images

Reverse Engineering

Tool	What it does	Where it appears	Roadmap link
radare2	Command-line reverse engineering framework	S2E12 — Tyrell Wellick reverse engineering a binary	Phase 3 — a lighter alternative/ companion to Ghidra, already in your resource list
Ghidra	Free NSA reverse-engineering suite (GUI)	Not shown on screen — but the show's era-accurate equivalent	Phase 3 — already your core tool for this phase
IDA Pro	Industry-standard commercial disassembler	S3E2 — Elliot patches a firmware installer's signature-check, shown in IDA's disassembly view	Phase 3 — paid tool; Ghidra is the free equivalent your roadmap already uses
binwalk	Extracts filesystems/data embedded in firmware images	S2 finale — Tyrell uses it with radare2 while analyzing the UPS firmware the Dark Army weaponized	Phase 3 — already your core Phase 3 tool

Vehicle / CAN Bus

Tool	What it does	Where it appears	Roadmap link
candump (can-utils)	Reads live CAN bus messages from a vehicle	S1E6 — used alongside Metasploit and BlueSniff to pull data from a police car computer	Phase 5 — natural extension once you finish RF & firmware phases; automotive security is a common Advanced-track specialization

© Hardware Devices Used In The Show

These are the physical gadgets, not software — this is the part that overlaps most directly with your roadmap, since several of these are things you'll effectively be *building the skills to use* (or building yourself) by the time you finish Phase 4.

Device	Show appearance	What it actually does	Roadmap link	Approx. EGP / Egypt notes
USB Rubber Ducky (Hak5)	S2E9 "Rubber Duckie, You're The One" — Angela uses it to harvest an E Corp exec's passwords	Looks like a USB flash drive, but registers as a keyboard and types a pre-scripted payload at superhuman speed (HID keystroke injection)	Phase 1 as a project add-on	Official Hak5 unit is import-only (~2,500–4,000 EGP shipped). Budget alternative: a **Raspberry Pi Pico** (200–350 EGP, already Amazon.eg) running the open-source "DuckyScript on Pico" firmware does the same HID trick for a fraction of the price.
Raspberry Pi (various models)	Used repeatedly by fsociety as small covert boxes for network implants	Full Linux computer small enough to hide inside a wall socket or facility and left running as a persistent foothold	Phase 2 / Phase 5 — as a glitch controller too	Raspberry Pi Pico already on your Phase 5 tool list (200–350 EGP); a full Raspberry Pi (Zero 2 W / 4) is Amazon.eg, ~1,200–3,500 EGP depending on model.
WiFi Pineapple (Hak5)	Referenced as part of the fsociety toolkit	Impersonates trusted WiFi networks to intercept traffic	Phase 4 concept	Import-only, ~6,000–9,000 EGP. Budget alternative: an **ESP32 Marauder** board (~300–600 EGP via AliExpress) covers the

	style — rogue-AP class device	from devices that auto-connect		same core WiFi recon/deauth concepts for learning purposes on networks you own.
SDR (Software Defined Radio)	General fsociety kit — receives/analyzes radio signals across a wide range	Turns a USB dongle + software into a radio receiver covering ~MHz to GHz ranges	Phase 4 — this is exactly your RTL-SDR	You already have this budgeted: RTL-SDR dongle, 450–700 EGP, Amazon.eg. The show-accurate "pro" version is a **HackRF One** (transmit + receive) — see upgrade table below.
Bluetooth sniffing rig (BlueSniff-class)	S1E6 — used with Metasploit to pull data from a car's onboard computer via Bluetooth	Passively collects information from Bluetooth-enabled devices without needing to pair	Phase 4 — your BLE sniffing week	Covered by your existing nRF52840 dongle (350–600 EGP) or a second cheap ESP32.

Not shown in the show, but the modern equivalent of its whole toolkit

Device	Covers	Approx. price	When to consider it
Flipper Zero	RFID/NFC, sub-GHz RF, BLE, GPIO, infrared — a portable all-in-one	~20,000+ EGP import	Your own roadmap already flags this correctly: not worth buying before Phase 4. It's a convenience toy that *replaces* tools you'll have already built skill with — buy it to save time later, not to skip learning.
Proxmark3	Serious RFID/NFC research and cloning, beyond what RC522 can do	~4,500–8,000 EGP import	Upgrade path from your RC522, listed in your existing Budget vs Professional table.
HackRF One	Wideband SDR with transmit capability (your RTL-SDR is receive-only)	~9,000–13,000 EGP import	Upgrade path from your RTL-SDR — only once you've hit a real project needing transmit.

Ⓓ Verified Deep-Dive: Straight From the Show's Own Consultant

> The most reliable source for what's real in Mr. Robot isn't a fan listicle — it's **Ryan Kazanciyan**, the show's actual Technical Consultant (Chief Security Architect at Tanium), who co-wrote the hacks with producer Kor Adana and then published an episode-by-episode breakdown series called "*Mr. Robot Disassembled*" on Medium. Everything in this section is drawn from that series, so you can trust it's accurate rather than guessed.

Episode	What actually happens	Real-world basis	Roadmap link
S2 finale	Tyrell reverse-engineers UPS (backup power) firmware that the Dark Army weaponized	Built entirely with **radare2** and **binwalk** — genuine open-source tools, used exactly as a real analyst would	Phase 3
S3E2 "undo.gz"	A hardware implant captures/manipulates a monitor's video output via its HDMI/USB ports, undetectable from the infected computer	Directly based on **"Monitor Darkly,"** real research presented at DEF CON 24 (2016) on vulnerable LCD display-controller firmware	Phase 3 — a real example of "firmware lives in more devices than you think"
S3E1 "power-saver-mode.h"	Web server compromised via an Apache Struts vulnerability plus an exposed Tomcat admin panel with default credentials	Same vulnerability class behind the 2017 Equifax breach — the writers didn't know that news would break the same year	Outside — web app security, adjacent field
S2E12 / referenced later	A backdoor called **rwwwshell** gives remote command-line access; Elliot deletes it with a single `shred` command	Real, publicly-available Perl reverse-shell tool — chosen deliberately because it's old and simple, matching a bare-bones victim system	Outside
S3E4 "runtime-error.r00"	The Dark Army compromises E Corp's Hardware Security Modules (HSMs) — devices that safeguard cryptographic signing keys	Inspired by a real breach the consultant personally investigated in 2013, where attackers targeted a company's code-signing infrastructure	Phase 5 concept — HSMs are the "final boss" of embedded security
S3 finale "shutdown -r"	Elliot cracks a LUKS-encrypted disk from Romero's keyloggers using a deliberately weakened, backdoored random-number generator planted earlier in the story	The LUKS cracking mechanics are real and reproducible; only the *speed* was compressed for TV — a real crack could take days to weeks	Phase 3

Source series: Ryan Kazanciyan, "Mr. Robot Disassembled," published on Medium throughout Season 3 (2017). Search that exact title to read the full episode-by-episode originals.

E How To Actually Learn These, Legally

> The show compresses months of work into a 5-minute scene. In real life, every tool above has a slow, boring, completely legal on-ramp. Use the roadmap you already have — this table just tells you *which week* to bring each show-tool in.

Tool / Device	Practice environment	First concrete exercise
USB Rubber Ducky / Pico HID	Your own laptop, offline	Write a DuckyScript payload that just opens Notepad and types a message — prove the concept before anything more ambitious
Shodan	shodan.io free tier	Search your own home router's public IP (with permission — it's yours) and see what it exposes
Wireshark / tcpdump	Your own home network	Capture and identify the handshake when a phone joins your WiFi
John the Ripper / Hashcat	Offline VM, hashes you generated yourself	Hash your own test passwords and time how long weak vs strong ones take to crack — the lesson is about password strength, not other people's accounts
radare2 / Ghidra	IoTGoat (already in your Phase 3 plan)	Already scheduled — Week 14–15 in your roadmap
candump / CAN bus	A CAN bus simulator (e.g. ICSim on GitHub) — never a real moving vehicle	Run ICSim locally and practice reading/injecting simulated CAN frames
RTL-SDR	Your own remote/doorbell	Already scheduled — Week 21–22 in your roadmap

New GitHub Stars For This Appendix

- **github.com/zardus/ctf-tools** — installer scripts for dozens of the CTF/RE tools referenced above
- **github.com/Uraken255/ICSim** — CAN bus instrument cluster simulator, safe candump practice
- **github.com/hak5/usbrubberducky-payloads** — official DuckyScript payload examples (read, don't blindly run)
- **github.com/justcallmekoko/ESP32Marauder** — open-source WiFi/BLE recon firmware for ESP32, budget Pineapple-class learning

> **One more time:** Shodan, Wireshark, hash-cracking, and CAN tools are all legal to *possess and learn*. What makes them illegal is using them against systems you don't own or lack permission to test — identical to the rule already in your roadmap's Phase 1 legal reminder. Keep every exercise on this page inside that boundary and you're on the same track as the show's real-life technical consultants, who are professional security researchers.

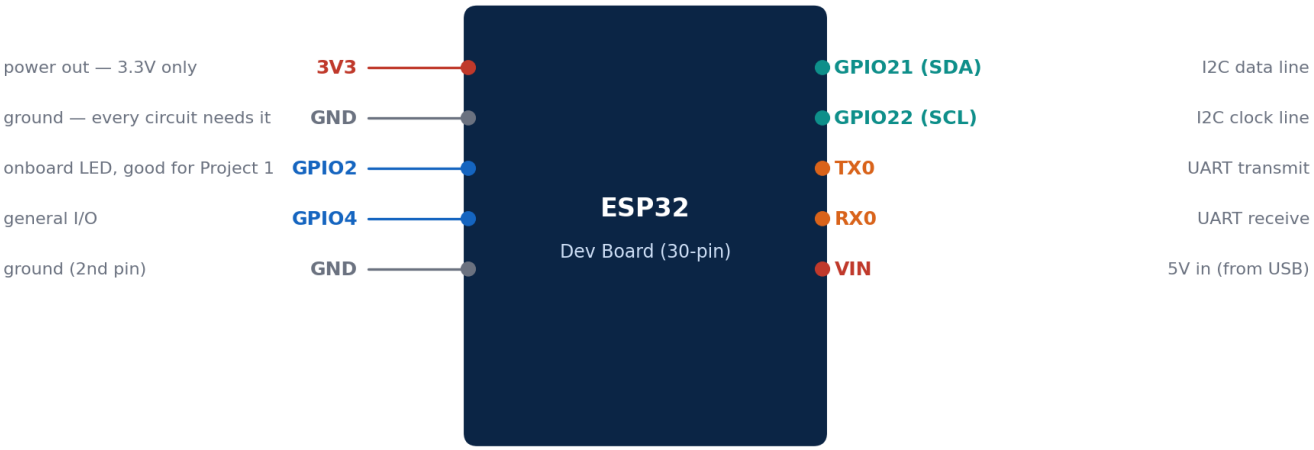
APPENDIX B

Visual Quick-Start Guide
Phase 1 Projects

Original wiring diagrams for your first four hands-on builds — drawn from scratch to match your exact ESP32 board and the exact GPIO pins used in this roadmap.

1 ESP32 Pin Reference

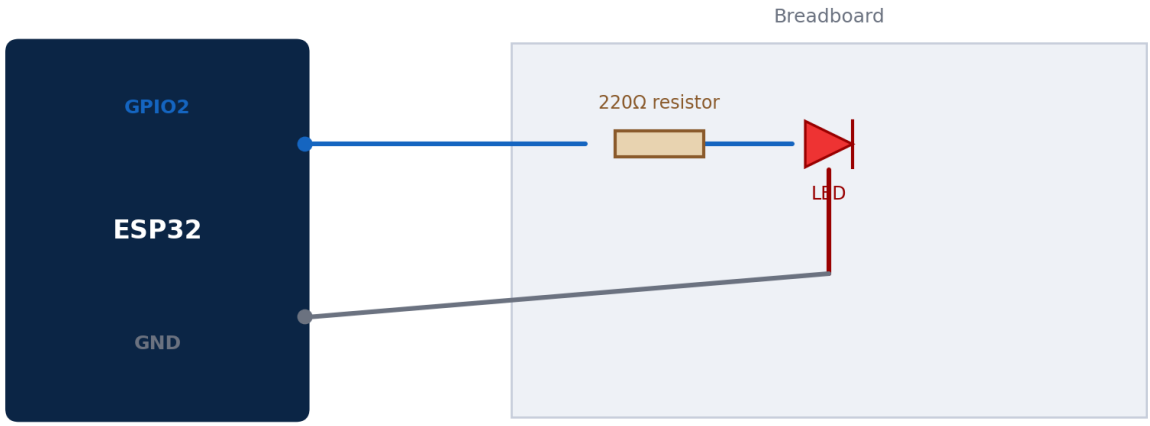
Keep this page open on your phone or a second screen every time you wire something in Phase 1. Every project below only ever uses these seven pins.



Rule: never wire a 5V signal into a GPIO pin — ESP32 logic is 3.3V only.

Before you wire anything: confirm your multimeter reads ~3.3V between the 3V3 pin and any GND pin while the board is powered via USB. If it doesn't, stop and check the USB cable/power first.

② Project 1 — Blink an LED (Week 1)



Path: GPIO2 → 220Ω resistor → LED (long leg / +) → LED (short leg / −) → GND

The resistor is not optional — without it, current will damage the LED or the GPIO pin.

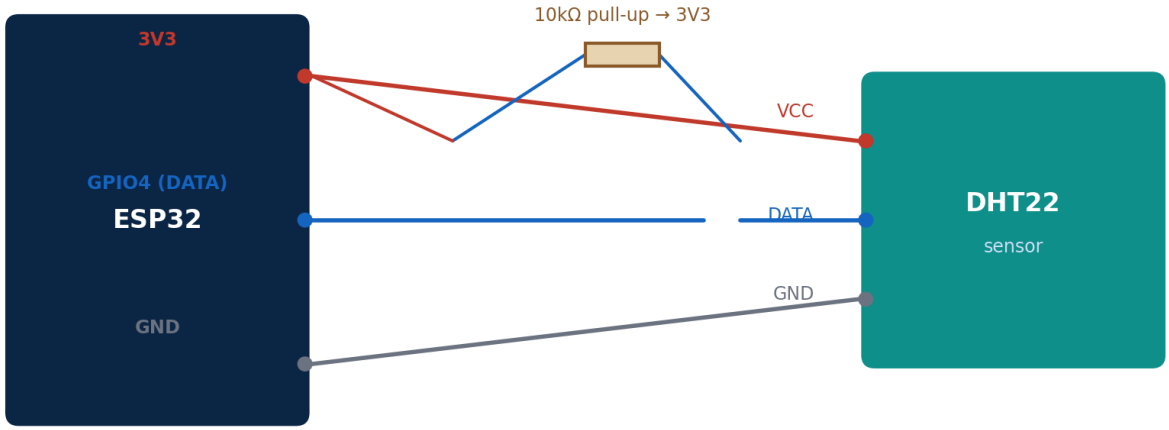
Parts needed: 1× LED (any color), 1× 220Ω resistor, 2× jumper wires, breadboard

Steps:

- Push the LED into the breadboard — long leg (anode/+) and short leg (cathode/−) in separate rows
- Push the resistor in series with the long leg
- Jumper wire from GPIO2 to the resistor's other end
- Jumper wire from the LED's short leg to any GND pin on the ESP32
- In Arduino IDE: File → Examples → 01.Basics → Blink, set the pin number to 2, then Upload

Sanity check: LED should blink roughly once per second. If it stays off, first check the LED isn't in backwards — flip it and try again before touching the code.

③ Project — DHT22 Sensor (Week 2)



Correction to note: DHT22 is NOT I2C — it uses its own single-wire digital protocol.

3 wires only: VCC, GND, and one DATA line (with a 10kΩ pull-up resistor to 3V3).

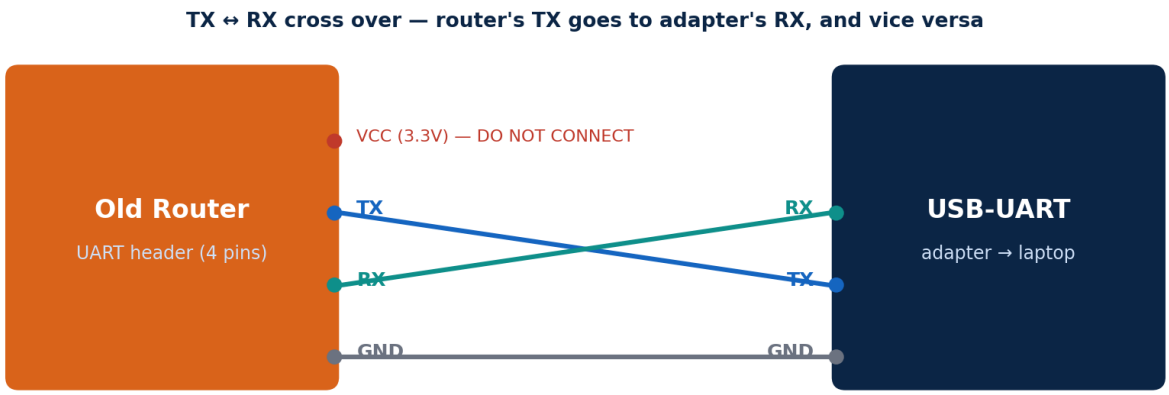
Parts needed: 1× DHT22 module, 1× 10kΩ resistor (often pre-installed on breakout boards — check yours first), 3× jumper wires

Steps:

- Wire VCC → 3V3, GND → GND, DATA → GPIO4
- If your DHT22 module is a bare 4-pin sensor (not a breakout board), you need the external 10kΩ pull-up resistor between DATA and 3V3 — most breakout boards already include it, check the silkscreen for a small resistor near the pins
- Install the "DHT sensor library" by Adafruit via Library Manager in Arduino IDE
- Load File → Examples → DHT sensor library → DHTtester, set pin to 4 and type to DHT22, Upload

Sanity check: Serial Monitor (115200 baud) should print a temperature and humidity reading every ~2 seconds. "NaN" readings usually mean a wiring or pull-up resistor issue, not a bad sensor.

④ Project — Router UART Boot Log (Week 5)



Critical: never connect the router's VCC pin. Power the router normally;

only TX, RX, and GND go to the USB-UART adapter.

Parts needed: old router (TP-Link/D-Link with visible header pins on the board), USB-UART adapter, multimeter, 3× jumper wires

Steps:

- Open the router, look for a 4-pin unpopulated header near the main chip — often silkscreened J1, J2, or UART
- With the router powered OFF, use multimeter continuity mode to find GND (continuity to the metal shield/chassis)
- Power on and use the multimeter in DC voltage mode: the pin sitting steady around 3.3V is VCC — leave it disconnected
- The remaining two pins are TX and RX — TX idles high and briefly dips when the router boots; if unsure, just try one way, and swap if you see garbage text
- Set your USB-UART terminal (e.g. PuTTY, screen, or Arduino IDE Serial Monitor) to 115200 baud first — most routers use this rate

Sanity check: readable boot log text (bootloader name, kernel messages) confirms correct wiring and baud rate. Garbled characters almost always mean wrong baud rate, not wrong pins.

⑤ Before You Start — Safety Checklist

- ☐ Multimeter confirms 3.3V on the 3V3 pin before wiring anything to it
- ☐ Never connect a 5V pin (VIN) directly to any GPIO — GPIOs are 3.3V-tolerant only
- ☐ Router/target device is unplugged from mains/router network — bench power only, never live traffic during pin-hunting
- ☐ Double-check GND is shared between every device in the circuit before powering on
- ☐ Only ever test hardware you personally own — same rule as the rest of this roadmap

When something doesn't work: re-check the wiring against the diagram first, then the baud rate, then the code — in that order. That order solves about 90% of Phase 1 problems. If you're still stuck, bring the exact error message back to this chat.

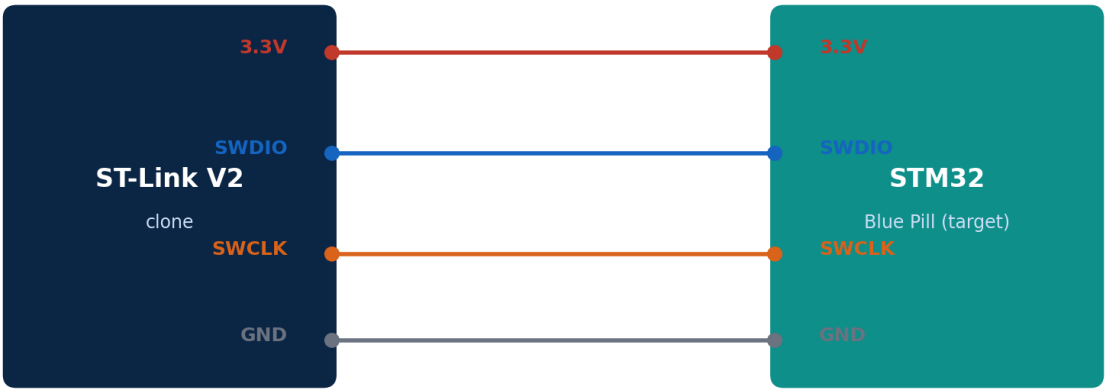
APPENDIX B — PART 2

Visual Quick-Start Guide
Phases 2 – 5

Wiring diagrams and step-by-step notes for every remaining hands-on project in your roadmap.

⑤ Phase 2, Project 1 — STM32 SWD Debug (Week 6–7)

4-wire SWD — faster to identify than full JTAG, standard on all ARM Cortex-M chips




```
Run:  openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
```

Confirm target voltage (3.3V here) before connecting — some boards run 1.8V logic.

Parts needed: ST-Link V2 clone, STM32 "Blue Pill" board, 4× jumper wires

Steps:

- Wire SWDIO, SWCLK, GND, and 3.3V exactly as shown — only 4 wires, no data pins needed
- Install OpenOCD (openocd.org)
- Run: `openocd -f interface/stlink.cfg -f target/stm32f1x.cfg`
- In a second terminal: `telnet localhost 4444`, then type `halt` to stop the CPU

Sanity check: OpenOCD's terminal output should say "Info: stm32f1x.cpu: hardware has 6 breakpoints" — that confirms the debugger sees the chip correctly.

⑥ Phase 2, Project 2 — Unlabelled Board Pin Hunt (Week 8–10)

① **Continuity test → GND**

② DC voltage test → VCC (steady voltage, leave disconnected)



③ **Logic analyzer on remaining 2 pins → capture at boot**

④ TX idles HIGH, dips briefly as text streams out at power-on

Parts needed: any board with an unmarked header, multimeter, USB logic analyzer, fine probe hooks

Workflow, in order:

- Power OFF the board — set multimeter to continuity mode, find GND against any metal shield or connector shell
- Power ON — set multimeter to DC voltage — the pin holding a steady 3.3V or 5V is VCC, leave it alone
- Clip the logic analyzer onto the 2 remaining unknown pins
- Power-cycle the board while capturing — look for the pin that's idling HIGH then briefly toggling as boot text streams out. That's TX.

Sanity check: if both remaining pins stay completely flat during boot, you likely have the wrong pins — recheck for a second header on the board.

⑦ Phase 3, Project 1 — Public Firmware Analysis (Week 12–13)

> **No wiring needed — this project is 100% software.**

Steps:

- Download 3 real firmware update files from any router/IoT manufacturer's support page
- Run: `binwalk -Me firmware.bin` (the `-M` flag recurses into nested archives — plain `-e` often misses embedded filesystems)
- Look inside the extracted folder for `etc/passwd`, `etc/shadow`, or any `.conf` files — these often contain hardcoded credentials

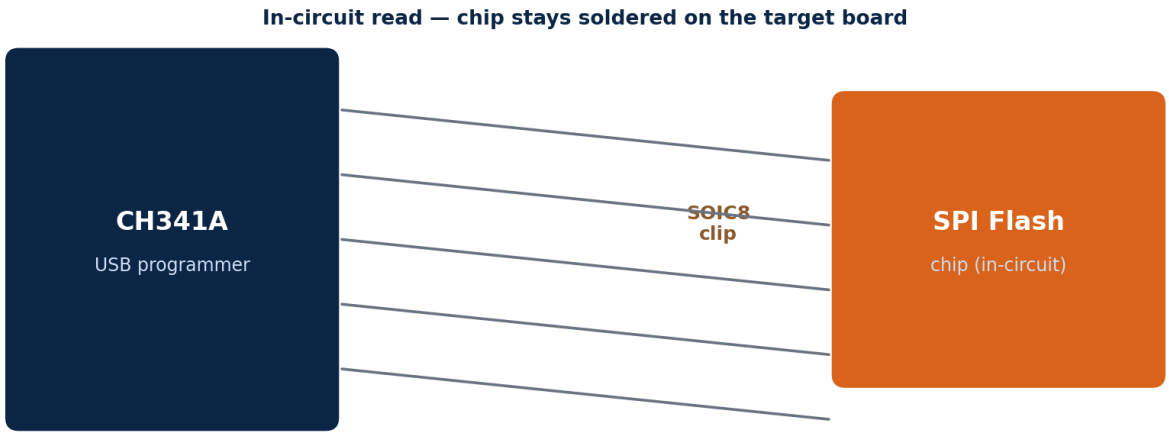
Sanity check: a successful extraction produces a `squashfs-root` or similar folder with a real Linux-style directory tree (bin, etc, usr) inside.

⑧ Phase 3, Project 2 — IoTGoat Walkthrough (Week 14–15)

No wiring needed — runs entirely inside a VM.

Download IoTGoat from github.com/OWASP/IoTGoat, load it into VirtualBox/QEMU as instructed in its README, then work through finding all 10 documented vulnerabilities. Document each one before moving to the next.

⑨ Phase 3, Project 3 — SPI Flash Dump (Week 16–17)



Check the exact chip's datasheet voltage FIRST — 3.3V into a 1.8V-only flash chip destroys it. Clip orientation dot must match pin-1 marking on the chip.

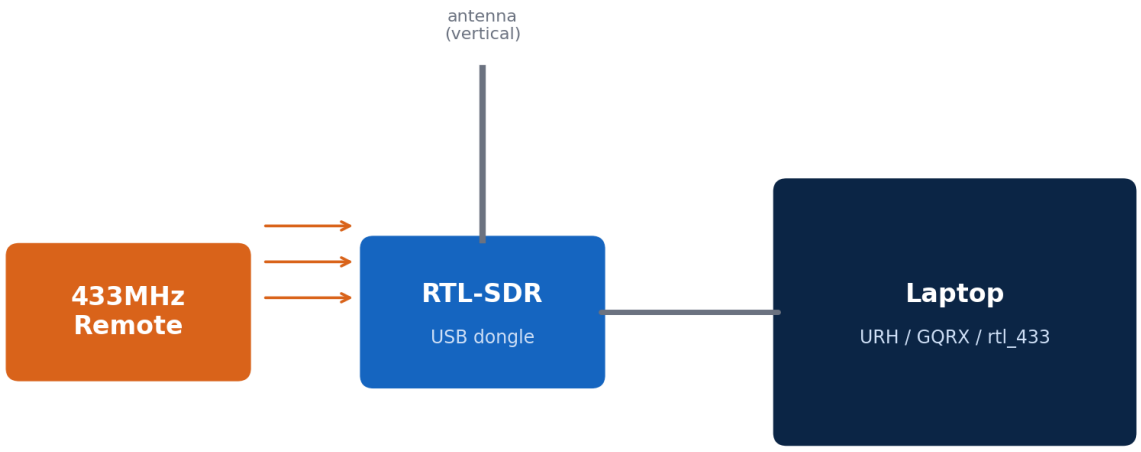
Parts needed: CH341A USB programmer, SOIC8 test clip, target device with visible SPI flash chip

Steps:

- Find the exact part number printed on the flash chip, search "[part number] datasheet" and confirm its operating voltage BEFORE clipping on
- Attach the SOIC8 clip — match the small dot/notch on the clip to pin 1 on the chip (usually marked with a dot on the chip itself)
- Run flashrom or the CH341A's bundled software to read the chip and save a `.bin` dump
- Verify with `binwalk dump.bin` — a real filesystem signature confirms a good read

Sanity check: a dump that's all `0xFF` or all `0x00` means the clip isn't seated correctly — re-check orientation before assuming the chip is bad.

⑩ Phase 4, Project 1 — 433MHz Remote Analysis (Week 20–22)



Capture 5–10 button presses, not just one — confirms the encoding is consistent

before you trust the decoded bit pattern.

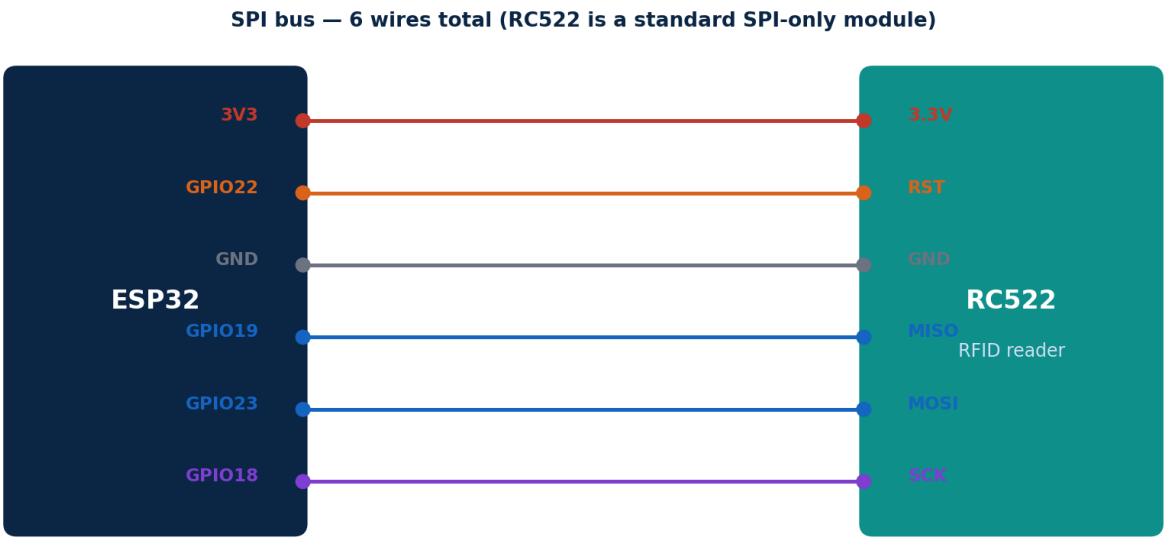
Parts needed: RTL-SDR dongle, a 433MHz remote you own (garage door, fan, doorbell)

Steps:

- Install Universal Radio Hacker (URH) — github.com/jopohl/urh
- Set center frequency near 433.92 MHz, start recording, then press the remote's button 5–10 times
- Use URH's auto-detect to identify modulation (usually ASK/OOK for cheap remotes) and decode the bit pattern

Sanity check: if all 5–10 captures decode to the exact same bit sequence, your decoding is correct — inconsistent results usually mean the sample rate is off.

⑪ Phase 4, Project 2 — RFID Card Read & Clone (Week 23–25)



RC522 is 3.3V ONLY — powering it from a 5V pin can permanently damage it.

Parts needed: RC522 module, ESP32, an RFID card/tag you own, T5577 writable card

Steps:

- Wire exactly as shown — RC522 is SPI, and it is **3.3V only**
- Install the "MFRC522" library in Arduino IDE, load the "DumpInfo" example
- Hold your card near the reader — the Serial Monitor will print the UID and any readable data blocks
- For 125kHz cards specifically, attempt a write to a T5577 blank using the matching library example

Sanity check: "Card UID:" printing consistently on every tap confirms good wiring; nothing printing usually means MOSI/MISO are swapped.

12 Phase 4, Project 3 — BLE Device Enumeration (Week 25–26)



Look for GATT characteristics that accept writes with no authentication

— that's the actual vulnerability class you're hunting for.

Parts needed: nRF52840 dongle (or a second ESP32), a BLE device you own (smart bulb, fitness band)

Steps:

- Install nRF Connect (mobile app) or nRF Connect for Desktop as the simplest starting point
- Scan for nearby devices, connect to yours, and browse its GATT services and characteristics
- Note which characteristics are marked "Write" or "Write Without Response" — try writing to one (safely, on your own device) and observe what changes

Sanity check: if you can toggle a bulb's state through a raw GATT write with no PIN/pairing prompt, you've just found the exact vulnerability class this exercise is teaching.

13 Phase 5, Project 1 — CTF Portfolio

No wiring needed — browser-based CTF challenges.

Work through Microcorruption (microcorruption.com) — a browser-based embedded assembly CTF — and the Riscure Rhme challenges. Write a structured report for each solve: challenge description, your approach, the vulnerability, the fix. Publish every write-up to your GitHub.

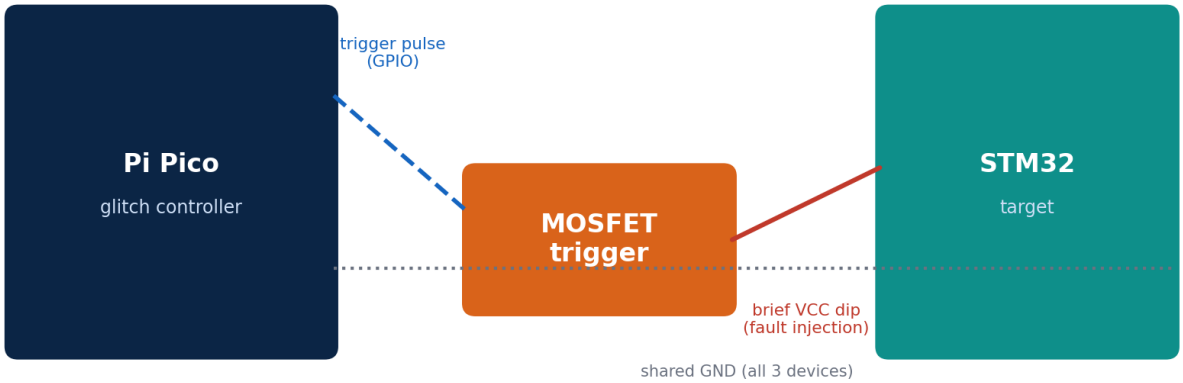
14 Phase 5, Project 2 — Full Device Assessment

No new wiring — this project combines every diagram in this guide.

Pick one device you own. Work through recon → UART/JTAG identification (diagrams 4 & 6) → firmware extraction (diagram 7) → static analysis (Ghidra/binwalk) → wireless assessment (diagrams 9 & 10, if applicable) → a full written report, using the Project Write-up Template already in your main roadmap.

⑮ Phase 5, Project 3 — Voltage Glitch Experiment (Week 34–36)

Concept: a precisely-timed power dip makes the target CPU skip one instruction — potentially bypassing a security check



Start with a wide timing sweep, then narrow down — most attempts just reset the target.

Parts needed: Raspberry Pi Pico, MOSFET + resistors (DIY glitch rig), STM32 Blue Pill target

Steps:

- Build the MOSFET trigger circuit following a documented STM32 RDP-bypass glitching tutorial (search the exact title — implementations vary by MOSFET part number)
- Wire the Pico's GPIO trigger output to the MOSFET gate, and route the target's VCC line through the MOSFET
- Share GND across all three boards — this is the single most common reason a glitch rig silently fails
- Sweep glitch timing/width systematically: start with a wide window, narrow down once you see any abnormal behavior

Sanity check: most attempts will just reset the target — that's expected and normal. Log every timing value you try and its result; the successful window is usually narrow and found through patient iteration, not luck.

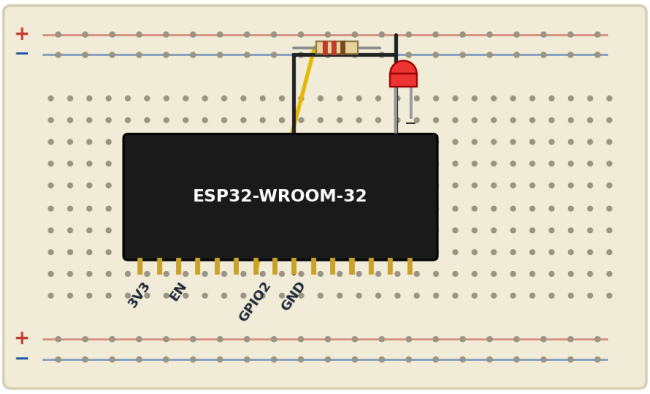
APPENDIX C

Realistic Breadboard Layouts

How the parts actually sit on the breadboard — physical placement, wire colors, and real component shapes, not just an abstract schematic.

① Project 1 — LED Blink, As It Actually Looks

Project 1 — LED Blink: Realistic Breadboard Placement



Yellow wire = GPIO2 signal • Black wire = ground return • Resistor sits between GPIO2 wire and LED's long leg

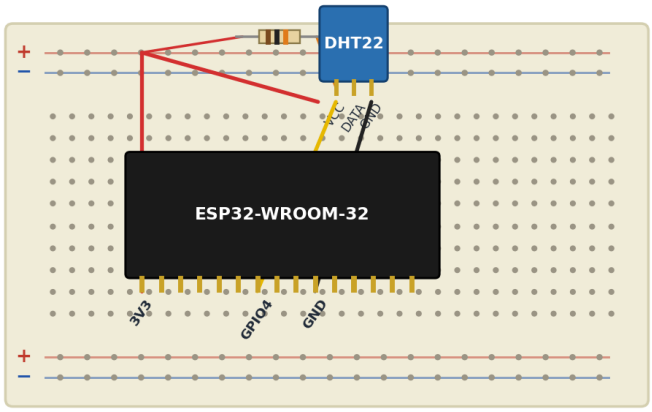
What you're seeing: the ESP32 straddles the breadboard's center gap (standard placement — its two pin rows land on separate strips so they don't short together). The resistor and LED sit in the row directly above GPIO2 and GND.

Wire color	From	To
Yellow	ESP32 GPIO2 pin	Resistor (then to LED long leg)
Black	ESP32 GND pin	LED short leg (via the blue "−" rail)

Physical tip: push each component's legs straight down into holes in the *same column strip* — breadboard columns are electrically connected vertically in groups of 5, so two legs in the same column are automatically wired together with no extra jumper needed.

② Project — DHT22 Sensor, As It Actually Looks

Project — DHT22 Sensor: Realistic Breadboard Placement



Red = 3V3 power • Yellow = GPIO4 data • Black = GND • Orange resistor = 10kΩ pull-up (DATA→3V3)

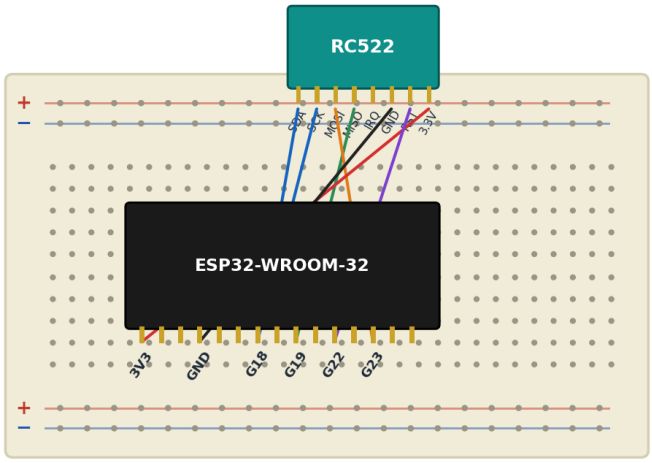
What you're seeing: the DHT22 module (most breakout boards look like a small blue rectangle, exactly like this) sits above the ESP32, straddling a few breadboard columns to keep its 3 pins separated.

Wire color	From	To
Red	ESP32 3V3	DHT22 VCC (and the pull-up resistor)
Yellow	ESP32 GPIO4	DHT22 DATA
Black	ESP32 GND	DHT22 GND
Orange resistor	DATA line	3V3 rail (10kΩ pull-up)

Physical tip: most DHT22 breakout boards (the blue PCB kind, not the bare white sensor) already have the pull-up resistor built in — check the underside of your board for a small resistor before adding a second one.

③ Project — RC522 RFID, As It Actually Looks

Project — RC522 RFID: Realistic Breadboard Placement



6 signal wires total — match each color to the labeled pin on both boards. RC522 needs 3.3V only.

What you're seeing: the RC522's 8-pin header sits above the ESP32. Only 6 of its 8 pins get used (SDA, SCK, MOSI, MISO, GND, RST, 3.3V) — the module has no VCC pin above 3.3V, so there's no risk of feeding it 5V by mistake if you wire it correctly.

Wire color	ESP32 pin	RC522 pin
Red	3V3	3.3V
Black	GND	GND
Blue	GPIO18	SCK / SDA
Green	GPIO19	MISO
Orange	GPIO23	MOSI
Purple	GPIO22	RST

Physical tip: because six wires cross close together, sleeve or twist matching-color wire pairs before plugging in — it makes it far easier to spot a wrong connection later, and matches how a real bench setup would look.

④ Reading Any Breadboard — The One Rule

The single most important thing about a breadboard: each short row of 5 holes on either side of the center gap is *already internally connected* — plugging two legs into the same row wires them together automatically, with no jumper needed. The two long rows top and bottom (marked + and −) run the *entire length* of the board and are meant for power and ground.

Once that one rule clicks, every diagram in this appendix — and every real breadboard you'll ever wire — reads the same way: find the row, drop the leg in, and trust the strip to carry the connection the rest of the way.

APPENDIX D

Component & Project Purpose Guide

What every project actually teaches you, and exactly what job each individual part is doing in the circuit — no part is in a diagram "just because."

① Phase 1 — Foundations

Project 1 — ESP32 Sensor Node (DHT22)

> **What this project actually teaches:** how any device reads information from the physical world (temperature, motion, light) and turns it into numbers your code can use. This is the foundation of every "sensor" in every IoT device you'll ever take apart.

Component	What it's actually doing here
ESP32	Runs your code and reads the sensor's signal on GPIO4
DHT22 sensor	Contains its own tiny internal circuit that measures temperature/humidity and encodes it as a timed digital signal on one wire
10kΩ resistor	Pull-up resistor — holds the DATA line at a known "high" voltage when the sensor isn't actively sending data, so the ESP32 can tell the difference between "no signal" and "signal = 0." Without it, the sensor often won't work at all, not just work poorly.
Breadboard + jumper wires	Temporary, solder-free connections so you can wire and rewire quickly while learning

Warm-up build — LED Blink (Week 1, before Project 1)

> **What this teaches:** the absolute simplest thing a microcontroller can do — turn one pin's voltage on and off under code control. Every other project in this roadmap is a more complex version of this same idea.

Component	What it's actually doing here
LED	A visual indicator only — proves your code reached the hardware and did something real. It has no deeper teaching purpose beyond "give visible proof."
220Ω resistor	Protects the LED (and the GPIO pin) by limiting current. An LED wired directly to GPIO2 with no resistor draws far more current than either part is rated for and can burn out one or both. This is the very first lesson in "always check the current path before connecting anything."

Neither the LED nor these two resistor values are listed in the main tool-budget table because they're normally bought together as a cheap beginner component kit (a bag of assorted resistors + a handful of LEDs) rather than sourced individually.

Project 2 — Router UART Boot Log

> **What this teaches:** how to extract information directly from a real device's internals using its debug port — the exact first step of any real hardware security assessment.

Component	What it's actually doing here
Old router	The target — its UART header is a debug interface manufacturers leave in for factory testing, usually undocumented in consumer manuals
Multimeter	Used *before* connecting anything — identifies which of the 4 unlabeled pins is GND (continuity test) and which is VCC (DC voltage test), so you never accidentally connect power wrong
USB-UART adapter	Translates the router's serial voltage signal into USB so your laptop can display it as readable text

② Phase 2 — Debug Interfaces

Project 1 — STM32 SWD Debug

> **What this teaches:** how to attach a professional debugger to a chip's CPU, stop it mid-execution, and inspect its memory directly — the core skill behind both legitimate firmware development and firmware security research.

Component	What it's actually doing here
ST-Link V2 clone	Translates the SWD debug protocol into USB so OpenOCD on your laptop can talk to the target chip's CPU
STM32 "Blue Pill"	A cheap, well-documented ARM target you're allowed to break — practicing "halt the CPU and read memory" here is safe before you try it on an unknown/valuable device

Project 2 — Unlabelled Board Pin Hunt

> **What this teaches:** a repeatable *methodology* for identifying unknown pins with no datasheet — because in real assessments, you rarely get a labeled header.

Component	What it's actually doing here
Multimeter	Two separate uses: continuity mode finds GND, DC voltage mode finds VCC — both before you risk touching anything with a logic analyzer or debugger
Logic analyzer	Once GND/VCC are ruled out, captures the remaining pins' activity during power-on to identify which one is TX (the one that idles high, then toggles as boot text streams out)

③ Phase 3 — Firmware & Reverse Engineering

Project 1 — Public Firmware Analysis (software only)

> **What this teaches:** firmware update files are just compressed filesystems — this project teaches you to open one up and read what's inside, the same way an analyst does before ever touching physical hardware.

Tool	What it's actually doing here
binwalk	Scans the firmware file for known file-format signatures and automatically extracts embedded filesystems, images, and compressed archives

Project 2 — IoTGoat Walkthrough (software only)

> **What this teaches:** what real embedded vulnerabilities actually look like in practice (hardcoded credentials, open debug services) — in a legal, deliberately-vulnerable practice target instead of a real product.

Project 3 — SPI Flash Dump

> **What this teaches:** extracting firmware directly from a chip when there's no other way in (no UART shell, no public update file) — a technique real researchers fall back on constantly.

Component	What it's actually doing here
CH341A programmer	Speaks the SPI protocol the flash chip expects and translates it to USB
SOIC8 test clip	Physically grips the flash chip's 8 legs without desoldering it from the board — "in-circuit" reading

④ Phase 4 — RF & Wireless Security

Project 1 — 433MHz Remote Analysis

> **What this teaches:** how simple wireless remotes (garage doors, fans, doorbells) encode their signal — and why many of them are trivially replayable.

Component	What it's actually doing here
RTL-SDR dongle	A radio receiver that hands raw radio-wave data to software instead of decoding it in hardware — that flexibility is what lets you analyze *any* frequency, not just one preset use

Project 2 — RFID Card Read & Clone

> **What this teaches:** how physical access-control systems (office badges, gym cards) work internally, and where their security actually breaks down.

Component	What it's actually doing here
RC522 module	Generates the RF field that powers a passive RFID card and reads back its response over SPI to the ESP32

T5577 writable card	A blank card that accepts being reprogrammed with another card's ID — used to test whether a target card has any anti-cloning protection

Project 3 — BLE Device Enumeration

> **What this teaches:** most "smart" devices (bulbs, bands, locks) expose a menu of remote-controllable functions over Bluetooth Low Energy — this project teaches you to read that menu and check whether it requires authentication.

Component	What it's actually doing here
nRF52840 dongle	A BLE radio capable of both scanning passively and actively connecting/enumerating a target device's services, which many built-in laptop Bluetooth chips can't do as flexibly

5 Phase 5 — Advanced / Professional

Project 1 — CTF Portfolio (software only)

> **What this teaches:** embedded-specific reverse-engineering and exploitation patterns, in a browser-based format built specifically for skill-building rather than a real target.

Project 2 — Full Device Assessment

> **What this teaches:** everything above, combined into one realistic, end-to-end professional engagement on a device you own — this is the project that actually proves you can do the job, not just isolated exercises.

Project 3 — Voltage Glitch Experiment

> **What this teaches:** fault injection — deliberately disturbing a chip's power to make it skip a security check it would normally enforce, such as a readout-protection lock.

Component	What it's actually doing here
Raspberry Pi Pico	Precisely times when the glitch fires, relative to the target's boot sequence — timing accuracy is the entire game in glitching
MOSFET trigger circuit	Acts as a fast electronic switch that briefly interrupts the target's power line on command from the Pico
STM32 target	The chip whose readout-protection you're attempting to bypass — same "Blue Pill" board from Phase 2, reused here for a more advanced attack

APPENDIX E

Corrections & Refinements

A technical review surfaced real gaps in the original roadmap. Every one confirmed here — verified, not just accepted — with the fix.

ⓘ Why This Appendix Exists

A second technical review (by Kimi, another AI assistant) of the original roadmap flagged 10 issues. Each one was independently checked — including a live web search to confirm the CH341A hardware flaw rather than taking the claim on faith — before being accepted here. This appendix documents what was correct, what needed nuance, and exactly what changes to make as you work through the roadmap.

1. DHT22 is not I2C

confirmed error

The error: Phase 1, Week 2 says "Wire a DHT22 sensor over I2C." This is wrong — the DHT22 uses its own proprietary single-wire digital protocol, not I2C. Appendix B (the visual guide) already wires it correctly and even flags this inconsistency, which is what exposed the error in the main text.

The fix: Two options — (a) keep the DHT22 and treat Week 2 as a single-wire protocol exercise instead of I2C, using the wiring in Appendix B exactly as shown, or (b) swap in a **BME280** or **AHT10** module instead — both are genuinely I2C (SDA/SCL, standard address scan works), similarly priced, and give you real I2C practice if that's specifically what Week 2 is meant to teach. Option (b) is recommended if I2C itself is the skill you want from this week.

2. Logic analyzer used in Phase 1, budgeted in Phase 2

confirmed inconsistency

The error: Phase 1 Project 1 asks you to "view the raw I2C bus traffic on your logic analyzer," but the logic analyzer only appears in the Phase 2 tools table.

The fix: Move the logic analyzer purchase (200–400 EGP) into your Phase 1 shopping list instead of Phase 2. It's cheap enough that buying it one phase early doesn't break the "buy tools on demand" principle, and you'll want it the moment you start Week 2's sensor work.

3. Python is missing entirely

valid gap

The gap: binwalk automation, URH scripting, and most real firmware-analysis tooling lean on Python. The roadmap only teaches C/C++.

The fix: Don't front-load it into Phase 1 — that would overload an already packed foundation phase. Instead, add a standing rule starting **Phase 3**: whenever a tool has a Python API (binwalk, URH, Ghidra's scripting console), spend 30–60 minutes learning to script it before moving on, rather than only using the GUI. By Phase 5 this compounds into real competency without ever needing a dedicated "learn Python" week.

4. WiFi security is absent from Phase 4

valid gap

The gap: Phase 4 covers sub-GHz, RFID, and BLE — but WiFi is the most common wireless protocol in real IoT devices, and it only appears via the ESP32 Marauder mention in Appendix A.

The fix: Add a 4th Phase 4 project: **WiFi Handshake Capture & Analysis**. Using aircrack-ng suite (already in your Appendix A software list) on your own home network, capture a WPA2 handshake when a device joins, and analyze it with Wireshark. This slots naturally after the BLE project (Week 26) and before the Phase 4 write-up week.

5. Linux/networking background assumed

valid, but lower priority for you specifically

The nuance: the review is right that Phase 3 leans on Linux CLI and VMs. For most beginners, a 2-week Linux primer before Phase 3 would be warranted. Given your day-to-day DevOps work already runs through Azure DevOps Server, self-hosted agents, and Windows/Linux mixed infrastructure, you're very likely past this gap already — treat this as a "check yourself, don't assume you need it" flag rather than a mandatory addition.

6. Phase 3 timeline is optimistic

valid

The concern: Weeks 12–20 assume you can go from "install Ghidra" to "analyze real bare-metal firmware" in 8 weeks. Real firmware without symbols, with a base address you have to figure out manually, is genuinely harder than the roadmap's pace implies.

The fix: No structural change needed — just expectation-setting. If Phase 3 takes 10–12 weeks instead of 8, that's normal, not falling behind. Don't compress the IoTGoat or Ghidra fundamentals to stay on the original schedule.

7. CH341A ships with a real hardware flaw confirmed, verified via web search

Independently verified: most CH341A programmer boards run their SPI data lines at 5V even when a 3.3V chip is inserted, because the CH341 chip itself is powered from USB 5V regardless of the target voltage jumper. This is a documented design flaw (confirmed across multiple independent hardware teardowns), not a rare defect — assume your unit has it unless proven otherwise.

The fix: before your first SPI flash dump in Phase 3, measure the voltage on the MISO and MOSI pins with your multimeter while a 3.3V chip is expected. If you read close to 5V instead of 3.3V, search "CH341A 3.3V fix" — it's a well-documented, reversible modification (cutting one trace and bridging to the onboard 3.3V regulator) that takes under 10 minutes and requires no special tools beyond a hobby knife.

8. Week numbering overlaps at the phase boundary confirmed, cosmetic

The fix: Phase 1 ends "Week 6," Phase 2 begins "Week 6" — read this as Phase 1 finishing by the end of week 6 and Phase 2 starting in week 7. No content changes needed, just don't let the shared number confuse your progress tracking.

9. Soldering is mentioned but never practiced valid

The gap: a soldering iron kit is on the Phase 1 shopping list, but no project actually requires using it — and real assessments eventually need desoldering/SMD rework.

The fix: Add an optional Week 6 exercise: solder a set of header pins onto a spare prototyping board, then desolder them cleanly. Cheap practice boards for this are sold everywhere electronics components are sold in Egypt. A basic hot air rework station becomes worth considering by Phase 3, once you're pulling chips for flash reading.

10. Side-channel analysis needs an oscilloscope valid

The gap: Phase 5 introduces power side-channel concepts, but without an oscilloscope to actually view power traces, this stays theoretical no matter how much you read.

The fix: Treat the side-channel section of Phase 5 as conceptual-only unless you add a scope. A basic used oscilloscope (even an old 2-channel analog one) is enough to get started and is far cheaper than a ChipWhisperer — check local electronics markets or Facebook Marketplace-style listings for used bench equipment before assuming you need the full professional setup.